

База данных 1.

Задание 1.

Ответ:

```
SELECT
  v.make,
  m.model
FROM Motorcycle m
JOIN Vehicle v ON v.model = m.model
WHERE m.horsepower > 150
AND m.price < 20000
AND m.type = 'Sport'
ORDER BY m.horsepower DESC;
```

The screenshot shows a PostgreSQL IDE window with a script editor and a results pane. The script editor contains the following SQL query:

```
SELECT
  v.make,
  m.model
FROM Motorcycle m
JOIN Vehicle v ON v.model = m.model
WHERE m.horsepower > 150
AND m.price < 20000
AND m.type = 'Sport'
ORDER BY m.horsepower DESC;
```

The results pane shows a table with 1 row and 2 columns. The columns are labeled 'make' and 'model'. The row contains the values 'Yamaha' and 'YZF-R1'.

make	model
Yamaha	YZF-R1

Задача 2.

Ответ:

```
SELECT
  v.make,
  c.model,
  c.horsepower,
  c.engine_capacity,
```

```
'Car' AS vehicle_type
FROM Car c
JOIN Vehicle v ON v.model = c.model
WHERE c.horsepower > 150
AND c.engine_capacity < 3
AND c.price < 35000
UNION ALL
SELECT
    v.maker,
    m.model,
    m.horsepower,
    m.engine_capacity,
    'Motorcycle' AS vehicle_type
FROM Motorcycle m
JOIN Vehicle v ON v.model = m.model
WHERE m.horsepower > 150
AND m.engine_capacity < 1.5
AND m.price < 20000
UNION ALL
SELECT
    v.maker,
    b.model,
    NULL AS horsepower,
    NULL AS engine_capacity,
    'Bicycle' AS vehicle_type
FROM Bicycle b
JOIN Vehicle v ON v.model = b.model
WHERE b.gear_count > 18
AND b.price < 4000
ORDER BY horsepower DESC NULLS LAST;
```

```

JOIN Vehicle v ON v.model = c.model
WHERE c.horsepower > 150
      AND c.engine_capacity < 3
      AND c.price < 35000

UNION ALL

-- SELECT
      v.make,
      m.model,
      m.horsepower,
      m.engine_capacity,
      'Motorcycle' AS vehicle_type
FROM Motorcycle m
JOIN Vehicle v ON v.model = m.model
WHERE m.horsepower > 150
      AND m.engine_capacity < 1.5
      AND m.price < 20000

UNION ALL

-- SELECT
      v.make,
      b.model,
      NULL AS horsepower,
      NULL AS engine_capacity,
      'Bicycle' AS vehicle_type
FROM Bicycle b
JOIN Vehicle v ON v.model = b.model
WHERE b.gear_count > 18
      AND b.price < 4000

ORDER BY horsepower DESC NULLS LAST;

```

Результат 1

SELECT v.make, c.model, c.horsepower, c.engine_capacity | Введите SQL выражение чтобы отфильтровать результаты

	A-Z maker	A-Z model	123 horsepower	123 engine_capacity	A-Z vehicle_type
1	Toyota	Camry	203	2,5	Car
2	Yamaha	YZF-R1	200	1	Motorcycle
3	Honda	Civic	158	2	Car
4	Trek	Domane	[NULL]	[NULL]	Bicycle
5	Giant	Defy	[NULL]	[NULL]	Bicycle

База данных 2.

Задача 1.

Ответ:

```

WITH stats AS (
  SELECT
    c.name AS car_name,
    c.class AS car_class,
    AVG(r.position) AS average_position,
    COUNT(*) AS race_count
  FROM Cars c
  JOIN Results r ON r.car = c.name
  GROUP BY c.name, c.class
)

```

```

),
best AS (
  SELECT
    car_class,
    MIN(average_position) AS min_avg
  FROM stats
  GROUP BY car_class
)
SELECT
  s.car_name,
  s.car_class,
  s.average_position,
  s.race_count
FROM stats s
JOIN best b
  ON s.car_class = b.car_class
 AND s.average_position = b.min_avg
ORDER BY s.average_position;

```

*postgres> Script X AZ name

```

WITH stats AS (
  SELECT
    c.name AS car_name,
    c.class AS car_class,
    AVG(r.position) AS average_position,
    COUNT(*) AS race_count
  FROM Cars c
  JOIN Results r ON r.car = c.name
  GROUP BY c.name, c.class
),
best AS (
  SELECT
    car_class,
    MIN(average_position) AS min_avg
  FROM stats
  GROUP BY car_class
)
SELECT
  s.car_name,
  s.car_class,
  s.average_position,
  s.race_count
FROM stats s
JOIN best b
  ON s.car_class = b.car_class
 AND s.average_position = b.min_avg
ORDER BY s.average_position;

```

cars 1 X

WITH stats AS (SELECT c.name AS car_name, c.class AS car_class, AVG(r.position) AS average_position, COUNT(*) AS race_count FROM Cars c JOIN Results r ON r.car = c.name GROUP BY c.name, c.class), best AS (SELECT car_class, MIN(average_position) AS min_avg FROM stats GROUP BY car_class) SELECT s.car_name, s.car_class, s.average_position, s.race_count FROM stats s JOIN best b ON s.car_class = b.car_class AND s.average_position = b.min_avg ORDER BY s.average_position;

	AZ car_name	AZ car_class	123 average_position	123 race_count
1	Ford Mustang	SportsCar	1	1
2	Ferrari 488	Convertible	1	1
3	Toyota RAV4	SUV	2	1
4	Mercedes-Benz S-Class	Luxury Sedan	2	1
5	BMW 3 Series	Sedan	3	1
6	Chevrolet Camaro	Coupe	4	1
7	Renault Clio	Hatchback	5	1
8	Ford F-150	Pickup	6	1

Задача 2.

Ответ:

```
WITH stats AS (  
    SELECT  
        c.name AS car_name,  
        c.class AS car_class,  
        AVG(r.position) AS average_position,  
        COUNT(*) AS race_count  
    FROM Cars c  
    JOIN Results r ON r.car = c.name  
    GROUP BY c.name, c.class  
)  
best AS (  
    SELECT  
        MIN(average_position) AS min_avg  
    FROM stats  
)  
SELECT  
    s.car_name,  
    s.car_class,  
    s.average_position,  
    s.race_count,  
    cl.country AS car_country  
FROM stats s  
JOIN best b ON s.average_position = b.min_avg  
JOIN Classes cl ON cl.class = s.car_class  
ORDER BY s.car_name  
LIMIT 1;
```

Script X AZ name

```

WITH stats AS (
    SELECT
        c.name AS car_name,
        c.class AS car_class,
        AVG(r.position) AS average_position,
        COUNT(*) AS race_count
    FROM Cars c
    JOIN Results r ON r.car = c.name
    GROUP BY c.name, c.class
),
best AS (
    SELECT
        MIN(average_position) AS min_avg
    FROM stats
)
SELECT
    s.car_name,
    s.car_class,
    s.average_position,
    s.race_count,
    cl.country AS car_country
FROM stats s
JOIN best b ON s.average_position = b.min_avg
JOIN Classes cl ON cl.class = s.car_class
ORDER BY s.car_name
LIMIT 1;

```

cars(+) 1 X

WITH stats AS (SELECT c.name AS car_name, c.class AS car_class, AVG(r.position) AS average_position, COUNT(*) AS race_count FROM Cars c JOIN Results r ON r.car = c.name GROUP BY c.name, c.class), best AS (SELECT MIN(average_position) AS min_avg FROM stats) SELECT s.car_name, s.car_class, s.average_position, s.race_count, cl.country AS car_country FROM stats s JOIN best b ON s.average_position = b.min_avg JOIN Classes cl ON cl.class = s.car_class ORDER BY s.car_name LIMIT 1;

	AZ car_name	AZ car_class	123 average_position	123 race_count	AZ car_country
1	Ferrari 488	Convertible	1	1	Italy

Задача 3.

Ответ:

```

WITH stats AS (
    SELECT
        c.name AS car_name,
        c.class AS car_class,
        AVG(r.position) AS average_position,
        COUNT(*) AS race_count
    FROM Cars c
    JOIN Results r ON r.car = c.name
    GROUP BY c.name, c.class
),
class_avg AS (
    SELECT
        car_class,
        AVG(average_position) AS class_avg_position,
        SUM(race_count) AS total_races
    FROM stats
)

```

```

GROUP BY car_class
),
best_classes AS (
SELECT
    MIN(class_avg_position) AS best_avg
FROM class_avg
)
SELECT
    s.car_name,
    s.car_class,
    s.average_position,
    s.race_count,
    cl.country AS car_country,
    ca.total_races
FROM stats s
JOIN class_avg ca ON ca.car_class = s.car_class
JOIN best_classes bc ON ca.class_avg_position = bc.best_avg
JOIN Classes cl ON cl.class = s.car_class
ORDER BY s.car_class, s.average_position;

```

*<postgres> Script X

```

WITH stats AS (
    SELECT
        c.name AS car_name,
        c.class AS car_class,
        AVG(r.position) AS average_position,
        COUNT(*) AS race_count
    FROM Cars c
    JOIN Results r ON r.car = c.name
    GROUP BY c.name, c.class
),
class_avg AS (
    SELECT
        car_class,
        AVG(average_position) AS class_avg_position,
        SUM(race_count) AS total_races
    FROM stats
    GROUP BY car_class
),
best_classes AS (
    SELECT
        MIN(class_avg_position) AS best_avg
    FROM class_avg
)
SELECT
    s.car_name,
    s.car_class,
    s.average_position,
    s.race_count,
    cl.country AS car_country,
    ca.total_races
FROM stats s
JOIN class_avg ca ON ca.car_class = s.car_class
JOIN best_classes bc ON ca.class_avg_position = bc.best_avg
JOIN Classes cl ON cl.class = s.car_class
ORDER BY s.car_class, s.average_position;

```

cars(+) 1 X

WITH stats AS (SELECT c.name AS car_name, c.class AS car_class, ... Введите SQL выражение чтобы отфильтровать результаты

	AZ car_name	AZ car_class	123 average_position	123 race_count	AZ car_country	123 total_races
1	Ferrari 488	Convertible	1	1	Italy	1
2	Ford Mustang	SportsCar	1	1	USA	1

Таблица

Текст

Задача 4.

Ответ:

```
WITH stats AS (  
  SELECT  
    c.name AS car_name,  
    c.class AS car_class,  
    AVG(r.position) AS average_position,  
    COUNT(*) AS race_count  
  FROM Cars c  
  JOIN Results r ON r.car = c.name  
  GROUP BY c.name, c.class  
)  
class_avg AS (  
  SELECT  
    car_class,  
    AVG(average_position) AS class_avg_position,  
    COUNT(*) AS car_count  
  FROM stats  
  GROUP BY car_class  
  HAVING COUNT(*) >= 2  
)  
SELECT  
  s.car_name,  
  s.car_class,  
  s.average_position,  
  s.race_count,  
  cl.country AS car_country  
FROM stats s  
JOIN class_avg ca ON ca.car_class = s.car_class  
JOIN Classes cl ON cl.class = s.car_class  
WHERE s.average_position < ca.class_avg_position  
ORDER BY s.car_class, s.average_position;
```


*<postgres> Script X

```

WITH stats AS (
    SELECT
        c.name AS car_name,
        c.class AS car_class,
        AVG(r.position) AS average_position,
        COUNT(*) AS race_count
    FROM Cars c
    JOIN Results r ON r.car = c.name
    GROUP BY c.name, c.class
),
class_avg AS (
    SELECT
        car_class,
        AVG(average_position) AS class_avg_position,
        COUNT(*) AS car_count
    FROM stats
    GROUP BY car_class
    HAVING COUNT(*) >= 2
)
SELECT
    s.car_name,
    s.car_class,
    s.average_position,
    s.race_count,
    cl.country AS car_country
FROM stats s
JOIN class_avg ca ON ca.car_class = s.car_class
JOIN Classes cl ON cl.class = s.car_class
WHERE s.average_position < ca.class_avg_position
ORDER BY s.car_class, s.average_position;

```

cars(+) 1 X

WITH stats AS (SELECT c.name AS car_name, c.class AS car_class, ... Введите SQL выражение чтобы отфильтровать результаты

	AZ car_name	AZ car_class	123 average_position	123 race_count	AZ car_country
1	BMW 3 Series	Sedan	3	1	Germany
2	Toyota RAV4	SUV	2	1	Japan

Задача 5.

Ответ:

```

WITH stats AS (
    SELECT
        c.name AS car_name,
        c.class AS car_class,
        AVG(r.position) AS average_position,
        COUNT(*) AS race_count
    FROM Cars c
    JOIN Results r ON r.car = c.name
    GROUP BY c.name, c.class
),
low_pos AS (
    SELECT
        car_class,

```

```

COUNT(*) AS low_position_count
FROM stats
WHERE average_position > 3.0
GROUP BY car_class
),
max_low AS (
SELECT MAX(low_position_count) AS max_count
FROM low_pos
),
class_races AS (
SELECT
    car_class,
    SUM(race_count) AS total_races
FROM stats
GROUP BY car_class
)
SELECT
    s.car_name,
    s.car_class,
    s.average_position,
    s.race_count,
    cl.country AS car_country,
    cr.total_races,
    lp.low_position_count
FROM stats s
JOIN low_pos lp ON lp.car_class = s.car_class
JOIN max_low ml ON ml.max_count = lp.low_position_count
JOIN class_races cr ON cr.car_class = s.car_class
JOIN Classes cl ON cl.class = s.car_class
ORDER BY lp.low_position_count DESC, s.car_class, s.average_position;

```

*<postgres> Script X

```

WITH stats AS (
    SELECT
        c.name AS car_name,
        c.class AS car_class,
        AVG(r.position) AS average_position,
        COUNT(*) AS race_count
    FROM Cars c
    JOIN Results r ON r.car = c.name
    GROUP BY c.name, c.class
),
low_pos AS (
    SELECT
        car_class,
        COUNT(*) AS low_position_count
    FROM stats
    WHERE average_position > 3.0
    GROUP BY car_class
),
max_low AS (
    SELECT MAX(low_position_count) AS max_count
    FROM low_pos
),
class_races AS (
    SELECT
        car_class,
        SUM(race_count) AS total_races
    FROM stats
    GROUP BY car_class
)
SELECT
    s.car_name,
    s.car_class,
    s.average_position,
    s.race_count,
    cl.country AS car_country,
    cr.total_races,
    lp.low_position_count
FROM stats s
JOIN low_pos lp ON lp.car_class = s.car_class

```

cars(+) 1 X

WITH stats AS (SELECT c.name AS car_name, c.class AS ca | Введите SQL выражение чтобы отфильтровать результаты

	AZ car_name	AZ car_class	123 average_position	123 race_count	AZ car_country	123 total_races
1	Chevrolet Camaro	Coupe	4	1	USA	1
2	Renault Clio	Hatchback	5	1	France	1
3	Ford F-150	Pickup	6	1	USA	1
4	BMW 3 Series	Sedan	3	1	Germany	2
5	Audi A4	Sedan	8	1	Germany	2

База данных 3.

Задача 1.

Ответ:

```

WITH booking_stats AS (
    SELECT
        c.ID_customer,
        c.name,
        c.email,
        c.phone,
        COUNT(*) AS total_bookings,
        COUNT(DISTINCT h.ID_hotel) AS unique_hotels,
        STRING_AGG(DISTINCT h.name, ', ') AS hotels_list,
        AVG(check_out_date - check_in_date) AS avg_stay
    FROM Booking b

```

```

JOIN Customer c ON c.ID_customer = b.ID_customer
JOIN Room r ON r.ID_room = b.ID_room
JOIN Hotel h ON h.ID_hotel = r.ID_hotel
GROUP BY c.ID_customer, c.name, c.email, c.phone
)
SELECT
    name,
    email,
    phone,
    total_bookings,
    hotels_list,
    avg_stay
FROM booking_stats
WHERE total_bookings > 2
AND unique_hotels > 1
ORDER BY total_bookings DESC;

```

*<postgres> Script × customer

```

WITH booking_stats AS (
    SELECT
        c.ID_customer,
        c.name,
        c.email,
        c.phone,
        COUNT(*) AS total_bookings,
        COUNT(DISTINCT h.ID_hotel) AS unique_hotels,
        STRING_AGG(DISTINCT h.name, ', ') AS hotels_list,
        AVG(check_out_date - check_in_date) AS avg_stay
    FROM Booking b
    JOIN Customer c ON c.ID_customer = b.ID_customer
    JOIN Room r ON r.ID_room = b.ID_room
    JOIN Hotel h ON h.ID_hotel = r.ID_hotel
    GROUP BY c.ID_customer, c.name, c.email, c.phone
)
SELECT
    name,
    email,
    phone,
    total_bookings,
    hotels_list,
    avg_stay
FROM booking_stats
WHERE total_bookings > 2
AND unique_hotels > 1
ORDER BY total_bookings DESC;

```

customer 1 ×

WITH booking_stats AS (SELECT c.ID_customer, c.name, c.email, c.phone, COUNT(*) AS total_bookings, COUNT(DISTINCT h.ID_hotel) AS unique_hotels, STRING_AGG(DISTINCT h.name, ', ') AS hotels_list, AVG(check_out_date - check_in_date) AS avg_stay FROM Booking b JOIN Customer c ON c.ID_customer = b.ID_customer JOIN Room r ON r.ID_room = b.ID_room JOIN Hotel h ON h.ID_hotel = r.ID_hotel GROUP BY c.ID_customer, c.name, c.email, c.phone) SELECT name, email, phone, total_bookings, hotels_list, avg_stay FROM booking_stats WHERE total_bookings > 2 AND unique_hotels > 1 ORDER BY total_bookings DESC; Введите SQL выражение чтобы отфильтровать результаты

	AZ name	AZ email	AZ phone	123 total_bookings	AZ hotels_list
1	Bob Brown	bob.brown@example.com	+2233445566	3	Grand Hotel, Ocean View Resort
2	Ethan Hunt	ethan.hunt@example.com	+5566778899	3	Mountain Retreat, Ocean View Resort

Задача 2.

Ответ:

```
WITH stats AS (  
    SELECT  
        c.ID_customer,  
        c.name,  
        COUNT(*) AS total_bookings,  
        COUNT(DISTINCT h.ID_hotel) AS unique_hotels,  
        SUM(r.price * (b.check_out_date - b.check_in_date)) AS total_spent  
    FROM Booking b  
    JOIN Customer c ON c.ID_customer = b.ID_customer  
    JOIN Room r ON r.ID_room = b.ID_room  
    JOIN Hotel h ON h.ID_hotel = r.ID_hotel  
    GROUP BY c.ID_customer, c.name  
)  
  
cond1 AS (  
    SELECT *  
    FROM stats  
    WHERE total_bookings > 2  
        AND unique_hotels > 1  
)  
  
cond2 AS (  
    SELECT *  
    FROM stats  
    WHERE total_spent > 500  
)  
  
SELECT  
    s.ID_customer,  
    s.name,  
    s.total_bookings,  
    s.total_spent,  
    s.unique_hotels  
FROM cond1 s  
JOIN cond2 USING (ID_customer)  
ORDER BY total_spent;
```



```

    AVG(r.price) AS avg_price,
    CASE
        WHEN AVG(r.price) < 175 THEN 'Дешевый'
        WHEN AVG(r.price) <= 300 THEN 'Средний'
        ELSE 'Дорогой'
    END AS hotel_type
FROM Hotel h
JOIN Room r ON r.ID_hotel = h.ID_hotel
GROUP BY h.ID_hotel, h.name
),
customer_hotels AS (
    SELECT
        c.ID_customer,
        c.name,
        STRING_AGG(DISTINCT h.name, ',') AS visited_hotels,
        MAX(CASE WHEN ha.hotel_type = 'Дорогой' THEN 3
            WHEN ha.hotel_type = 'Средний' THEN 2
            ELSE 1 END) AS pref_rank
    FROM Booking b
    JOIN Customer c ON c.ID_customer = b.ID_customer
    JOIN Room r ON r.ID_room = b.ID_room
    JOIN hotel_avg ha ON ha.ID_hotel = r.ID_hotel
    JOIN Hotel h ON h.ID_hotel = r.ID_hotel
    GROUP BY c.ID_customer, c.name
),
final AS (
    SELECT
        ID_customer,
        name,
        CASE
            WHEN pref_rank = 3 THEN 'Дорогой'
            WHEN pref_rank = 2 THEN 'Средний'
            ELSE 'Дешевый'
        END AS preferred_hotel_type,
        visited_hotels,
        pref_rank
    FROM customer_hotels
)
SELECT
    ID_customer,
    name,
    preferred_hotel_type,
    visited_hotels
FROM final
ORDER BY pref_rank, name;

```

*<postgres> Script X customer

```

FROM hotel h
JOIN Room r ON r.ID_hotel = h.ID_hotel
GROUP BY h.ID_hotel, h.name
),
customer_hotels AS (
SELECT
c.ID_customer,
c.name,
STRING_AGG(DISTINCT h.name, ',') AS visited_hotels,
MAX(CASE WHEN ha.hotel_type = 'Дорогой' THEN 3
        WHEN ha.hotel_type = 'Средний' THEN 2
        ELSE 1 END) AS pref_rank
FROM Booking b
JOIN Customer c ON c.ID_customer = b.ID_customer
JOIN Room r ON r.ID_room = b.ID_room
JOIN hotel_avg ha ON ha.ID_hotel = r.ID_hotel
JOIN Hotel h ON h.ID_hotel = r.ID_hotel
GROUP BY c.ID_customer, c.name
),
final AS (
SELECT
ID_customer,
name,
CASE
WHEN pref_rank = 3 THEN 'Дорогой'
WHEN pref_rank = 2 THEN 'Средний'
ELSE 'Дешевый'
END AS preferred_hotel_type

```

customer 1 X

WITH hotel_avg AS (SELECT h.ID_hotel, h.name, AVG(h.hotel_type) AS avg_hotel_type FROM hotel h GROUP BY h.ID_hotel, h.name)

	id_customer	name	preferred_hotel_type	visited_hotels
1	10	Hannah Montana	Дешевый	City Center Inn
2	3	Alice Johnson	Средний	Grand Hotel
3	4	Bob Brown	Средний	Grand Hotel,Ocean View Resort
4	5	Charlie White	Средний	Ocean View Resort
5	6	Diana Prince	Средний	Ocean View Resort
6	2	Jane Smith	Средний	Grand Hotel
7	1	John Doe	Средний	City Center Inn,Grand Hotel
8	7	Ethan Hunt	Дорогой	Mountain Retreat,Ocean View Resort
9	8	Fiona Apple	Дорогой	Mountain Retreat
10	9	George Washington	Дорогой	City Center Inn,Mountain Retreat

База данных 4.

Задача 1.

Ответ:

WITH RECURSIVE subordinates AS (

```

SELECT
e.EmployeeID,
e.Name,

```



```

    e.ManagerID,
    e.DepartmentID,
    e.RoleID
FROM Employees e
WHERE e.EmployeeID = 1
UNION ALL

SELECT
    e.EmployeeID,
    e.Name,
    e.ManagerID,
    e.DepartmentID,
    e.RoleID
FROM Employees e
JOIN subordinates s ON e.ManagerID = s.EmployeeID
),
project_data AS (
    SELECT
        p.DepartmentID,
        STRING_AGG(p.ProjectName, ', ' ORDER BY p.ProjectName) AS ProjectNames
    FROM Projects p
    GROUP BY p.DepartmentID
),
task_data AS (
    SELECT
        t.AssignedTo,
        STRING_AGG(t.TaskName, ', ' ORDER BY t.TaskName) AS TaskNames
    FROM Tasks t
    GROUP BY t.AssignedTo
)
SELECT
    s.EmployeeID,
    s.Name AS EmployeeName,
    s.ManagerID,
    d.DepartmentName,
    r.RoleName,
    pd.ProjectNames,
    td.TaskNames
FROM subordinates s
LEFT JOIN Departments d ON d.DepartmentID = s.DepartmentID
LEFT JOIN Roles r ON r.RoleID = s.RoleID
LEFT JOIN project_data pd ON pd.DepartmentID = s.DepartmentID
LEFT JOIN task_data td ON td.AssignedTo = s.EmployeeID
ORDER BY s.Name;

```

*<postgres> Script X

```

task_data AS (
    SELECT
        t.AssignedTo,
        STRING_AGG(t.TaskName, ' ' ORDER BY t.TaskName) AS TaskNames
    FROM Tasks t
    GROUP BY t.AssignedTo
)

SELECT
    s.EmployeeID,

```

departments(+) 1 X

WITH RECURSIVE subordinates AS (SELECT e.EmployeeID, e.Name, e.ManagerID, e.DepartmentID, e.RoleID FROM Employees e WHERE e.EmployeeID = 1 UNION ALL SELECT e.EmployeeID, e.Name, e.ManagerID, e.DepartmentID, e.RoleID FROM Employees e JOIN subordinates s ON e.ManagerID = s.EmployeeID),

	123 employeeid	AZ employeename	123 managerid	AZ departmentname	AZ rolename	AZ projectnames	AZ tasknames
1	20	Александр Александров	3	Отдел маркетинга	Менеджер	Проект В	[NULL]
2	4	Алексей Алексеев	2	Отдел продаж	Менеджер	Проект А	Задача 1: Подготовка отчета по продажам
3	16	Анастасия Анастасиева	7	Отдел поддержки	Специалист по поддержке	Проект D	[NULL]
4	29	Анастасия Анастасиевна	7	Отдел поддержки	Специалист по поддержке	Проект D	[NULL]
5	6	Андрей Андреев	1	Отдел разработки	Разработчик	Проект С	Задача 15: Обновление сайта, Задача 16: Внедрение нового функционала
6	30	Валентин Валентинов	6	Отдел разработки	Разработчик	Проект С	[NULL]
7	15	Виктор Викторов	4	Отдел продаж	Менеджер	Проект А	[NULL]
8	21	Галина Галина	7	Отдел поддержки	Специалист по поддержке	Проект D	[NULL]
9	26	Денис Денисов	6	Отдел разработки	Разработчик	Проект С	[NULL]
10	14	Дмитрий Дмитриев	3	Отдел маркетинга	Маркетолог	Проект В	[NULL]
11	25	Екатерина Екатеринина	7	Отдел поддержки	Специалист по поддержке	Проект D	[NULL]
12	7	Елена Еленова	1	Отдел поддержки	Специалист по поддержке	Проект D	Задача 12: Настройка системы поддержки клиентов
13	1	Иван Иванов	[NULL]	Отдел продаж	Генеральный директор	Проект А	[NULL]
14	28	Игорь Игорев	2	Отдел продаж	Менеджер	Проект А	[NULL]
15	11	Ирина Ирина	6	Отдел разработки	Разработчик	Проект С	Задача 8: Тестирование нового продукта
16	13	Кристина Кристина	4	Отдел продаж	Менеджер	Проект А	[NULL]
17	18	Людмила Людмила	3	Отдел маркетинга	Маркетолог	Проект В	[NULL]
18	17	Максим Максимов	6	Отдел разработки	Разработчик	Проект С	[NULL]
19	23	Марина Маринина	3	Отдел маркетинга	Маркетолог	Проект В	[NULL]
20	5	Мария Мариева	3	Отдел маркетинга	Менеджер	Проект В	Задача 5: Создание рекламной кампании
21	19	Наталья Наталья	4	Отдел продаж	Менеджер	Проект А	[NULL]
22	10	Николай Николаев	6	Отдел разработки	Разработчик	Проект С	Задача 11: Интеграция с новым API, Задача 13: Оптимизация базы данных
23	8	Олег Олегов	2	Отдел продаж	Менеджер	Проект А	Задача 7: Проведение тренинга для клиентов
24	27	Ольга Ольгина	3	Отдел маркетинга	Маркетолог	Проект В	[NULL]
25	22	Павел Павлов	6	Отдел разработки	Разработчик	Проект С	[NULL]
26	2	Петр Петров	1	Отдел продаж	Директор	Проект А	[NULL]
27	3	Светлана Светлова	1	Отдел маркетинга	Директор	Проект В	[NULL]
28	12	Сергей Сергеев	7	Отдел поддержки	Специалист по поддержке	Проект D	Задача 4: Поддержка клиентов, Задача 6: Внедрение нового ПО
29	24	Станислав Станиславов	4	Отдел продаж	Менеджер	Проект А	[NULL]
30	9	Татьяна Татеева	3	Отдел маркетинга	Маркетолог	Проект В	Задача 10: Подготовка маркетинговой стратегии

Задача 2.

Ответ:

WITH RECURSIVE subordinates AS (

SELECT

e.EmployeeID,

e.Name,

e.ManagerID,

e.DepartmentID,

e.RoleID

FROM Employees e

WHERE e.EmployeeID = 1

UNION ALL

SELECT

e.EmployeeID,

e.Name,

e.ManagerID,

e.DepartmentID,

e.RoleID

FROM Employees e

JOIN subordinates s ON e.ManagerID = s.EmployeeID

),

```

project_data AS (
    SELECT
        p.DepartmentID,
        STRING_AGG(p.ProjectName, ', ' ORDER BY p.ProjectName) AS ProjectNames
    FROM Projects p
    GROUP BY p.DepartmentID
),

```

```

task_data AS (
    SELECT
        t.AssignedTo,
        STRING_AGG(t.TaskName, ', ' ORDER BY t.TaskName) AS TaskNames,
        COUNT(*) AS TotalTasks
    FROM Tasks t
    GROUP BY t.AssignedTo
),

```

```

direct_subs AS (
    SELECT
        ManagerID,
        COUNT(*) AS TotalSubordinates
    FROM Employees
    GROUP BY ManagerID
)

```

```

SELECT
    s.EmployeeID,
    s.Name AS EmployeeName,
    s.ManagerID,
    d.DepartmentName,
    r.RoleName,
    pd.ProjectNames,
    td.TaskNames,
    COALESCE(td.TotalTasks, 0) AS TotalTasks,
    COALESCE(ds.TotalSubordinates, 0) AS TotalSubordinates
FROM subordinates s
LEFT JOIN Departments d ON d.DepartmentID = s.DepartmentID
LEFT JOIN Roles r ON r.RoleID = s.RoleID
LEFT JOIN project_data pd ON pd.DepartmentID = s.DepartmentID
LEFT JOIN task_data td ON td.AssignedTo = s.EmployeeID
LEFT JOIN direct_subs ds ON ds.ManagerID = s.EmployeeID
ORDER BY s.Name;

```

postgres> Script X projects roles tasks

```

WITH RECURSIVE subordinates AS (
    -- Иван Иванович
    SELECT
        e.EmployeeID,
        e.Name,
        e.ManagerID,
        e.DepartmentID,
        e.RoleID
    FROM Employees e
    WHERE e.EmployeeID = 1

    UNION ALL

    -- Все подчинённые и подчинённые подчинённых
    SELECT
        e.EmployeeID,
        e.Name,
        e.ManagerID,
        e.DepartmentID,
        e.RoleID
    FROM Employees e
    JOIN subordinates s ON e.ManagerID = s.EmployeeID
),

-- Проекты привязаны к отделам
project_data AS (
    SELECT

```

departments(*) 1 X

	123 employeeid	AZ employeename	123 managerid	AZ departmentname	AZ rolename	AZ projectnames	AZ tasknames	123 totaltasks	123 totalsubordinates
14	28	Игорь Игорев		Отдел продаж	Менеджер	Проект А	(NULL)	0	0
15	11	Ирина Ирина		Отдел разработки	Разработчик	Проект С	Задача 8: Тестировани	1	0
16	13	Кристина Кристинина		Отдел продаж	Менеджер	Проект А	(NULL)	0	0
17	18	Людмила Людмила		Отдел маркетинга	Маркетолог	Проект В	(NULL)	0	0
18	17	Максим Максимов		Отдел разработки	Разработчик	Проект С	(NULL)	0	0
19	23	Марина Маринина		Отдел маркетинга	Маркетолог	Проект В	(NULL)	0	0
20	5	Мария Мариева		Отдел маркетинга	Менеджер	Проект В	Задача 5: Создание ре	1	0
21	19	Наталья Наталья		Отдел продаж	Менеджер	Проект А	(NULL)	0	0
22	10	Николай Николаев		Отдел разработки	Разработчик	Проект С	Задача 11: Интеграция	2	0
23	8	Олег Олегов		Отдел продаж	Менеджер	Проект А	Задача 7: Проведение	1	0
24	27	Ольга Ольгина		Отдел маркетинга	Маркетолог	Проект В	(NULL)	0	0
25	22	Павел Павлов		Отдел разработки	Разработчик	Проект С	(NULL)	0	0
26	2	Петр Петров		Отдел продаж	Директор	Проект А	(NULL)	0	3
27	3	Светлана Светлова		Отдел маркетинга	Директор	Проект В	(NULL)	0	7
28	12	Сергей Сергеев		Отдел поддержки	Специалист по подде	Проект D	Задача 4: Поддержка	2	0
29	24	Станислав Станислав		Отдел продаж	Менеджер	Проект А	(NULL)	0	0
30	9	Татьяна Татеева		Отдел маркетинга	Маркетолог	Проект В	Задача 10: Подготовка	3	0

Задача 3.

Ответ:

```

WITH RECURSIVE all_subs AS (
    SELECT
        e.EmployeeID AS root_manager,
        e.EmployeeID,
        e.ManagerID
    FROM Employees e
    UNION ALL

    SELECT
        a.root_manager,
        e.EmployeeID,
        e.ManagerID
    FROM Employees e
    JOIN all_subs a ON e.ManagerID = a.EmployeeID
),

sub_count AS (
    SELECT
        root_manager AS EmployeeID,
        COUNT(*) - 1 AS TotalSubordinates
    FROM all_subs
    GROUP BY root_manager
),

project_data AS (

```

```

SELECT
    p.DepartmentID,
    STRING_AGG(p.ProjectName, ', ' ORDER BY p.ProjectName) AS ProjectNames
FROM Projects p
GROUP BY p.DepartmentID
),
task_data AS (
    SELECT
        t.AssignedTo,
        STRING_AGG(t.TaskName, ', ' ORDER BY t.TaskName) AS TaskNames
    FROM Tasks t
    GROUP BY t.AssignedTo
)
SELECT
    e.EmployeeID,
    e.Name AS EmployeeName,
    e.ManagerID,
    d.DepartmentName,
    r.RoleName,
    pd.ProjectNames,
    td.TaskNames,
    sc.TotalSubordinates
FROM Employees e
JOIN Roles r ON r.RoleID = e.RoleID
JOIN Departments d ON d.DepartmentID = e.DepartmentID
LEFT JOIN project_data pd ON pd.DepartmentID = e.DepartmentID
LEFT JOIN task_data td ON td.AssignedTo = e.EmployeeID
JOIN sub_count sc ON sc.EmployeeID = e.EmployeeID
WHERE r.RoleName = 'Менеджер'
AND sc.TotalSubordinates > 0
ORDER BY e.Name;

```

